MA 551 - Homework 2

Adam Field

March 5, 2026

1 Problem 1: Levenberg-Marquardt for Old Faithful

The Old Faithful eruption durations are modeled as a two-component Gaussian mixture:

$$f(x \mid \mu_1, \mu_2) = p \cdot \phi(x;\, \mu_1, \sigma_1) + (1-p) \cdot \phi(x;\, \mu_2, \sigma_2),$$

where $\phi(x;\, \mu, \sigma)$ is the Gaussian density. The parameters p, σ_1, and σ_2 are estimated from the data by a preliminary clustering at the natural break point of 3 minutes; μ_1 and μ_2 are then the free parameters over which the log-likelihood is maximized.

```r
data(faithful)
x_erupt <- faithful$eruptions

# Estimate fixed parameters from data
short_mask <- x_erupt < 3
p_hat      <- mean(short_mask)
sigma1     <- sd(x_erupt[short_mask])
sigma2     <- sd(x_erupt[!short_mask])

# Log-likelihood as a function of mu = c(mu1, mu2)
log_lik <- function(mu) {
  comp1 <- p_hat       * dnorm(x_erupt, mu[1], sigma1)
  comp2 <- (1 - p_hat) * dnorm(x_erupt, mu[2], sigma2)
  sum(log(comp1 + comp2 + 1e-300))
}

# Numerical gradient (central differences)
num_grad <- function(f, theta, h = 1e-5) {
  sapply(seq_along(theta), function(j) {
    e <- replace(numeric(length(theta)), j, h)
    (f(theta + e) - f(theta - e)) / (2 * h)
  })
}

# Numerical Hessian (central differences)
num_hess <- function(f, theta, h = 1e-4) {
  n <- length(theta)
  H <- matrix(0, n, n)
  for (j in seq_len(n)) for (k in seq_len(n)) {
    ej <- replace(numeric(n), j, h)
    ek <- replace(numeric(n), k, h)
    H[j, k] <- (f(theta+ej+ek) - f(theta+ej-ek) -
                f(theta-ej+ek) + f(theta-ej-ek)) / (4 * h^2)
  }
  H
}
```

The Levenberg-Marquardt algorithm for maximization proceeds as follows. At each step the proposed update is

$$\Delta\theta = \big(-H(\theta) + \lambda I\big)^{-1} \nabla \ell(\theta),$$

where $\lambda > 0$ damps the step toward the gradient-ascent direction when the Newton step overshoots. If the proposal improves the log-likelihood it is accepted and λ is halved; otherwise it is rejected and λ is doubled.

```r
LevenbergMarquardt <- function(loglik_fn, theta0, lambda = 0.1,
                               tol = 1e-8, maxiter = 500) {
  theta   <- theta0
  history <- matrix(theta, nrow = 1)

  for (iter in seq_len(maxiter)) {
    g    <- num_grad(loglik_fn, theta)
    H    <- num_hess(loglik_fn, theta)
    step <- solve(-H + lambda * diag(length(theta)), g)

    if (loglik_fn(theta + step) > loglik_fn(theta)) {
      theta  <- theta + step
      lambda <- lambda / 2
    } else {
      lambda <- lambda * 2
    }
    history <- rbind(history, theta)

    if (max(abs(step)) < tol) break
  }

  list(theta = theta, history = history, iter = iter)
}
```

```r
result <- LevenbergMarquardt(log_lik, theta0 = c(2, 3))

cat(sprintf("MLE: mu1 = %.4f, mu2 = %.4f\n",
            result$theta[1], result$theta[2]))
```

```
## MLE: mu1 = 2.0304, mu2 = 4.2842
```

```r
cat(sprintf("Log-likelihood at MLE: %.4f\n", log_lik(result$theta)))
```

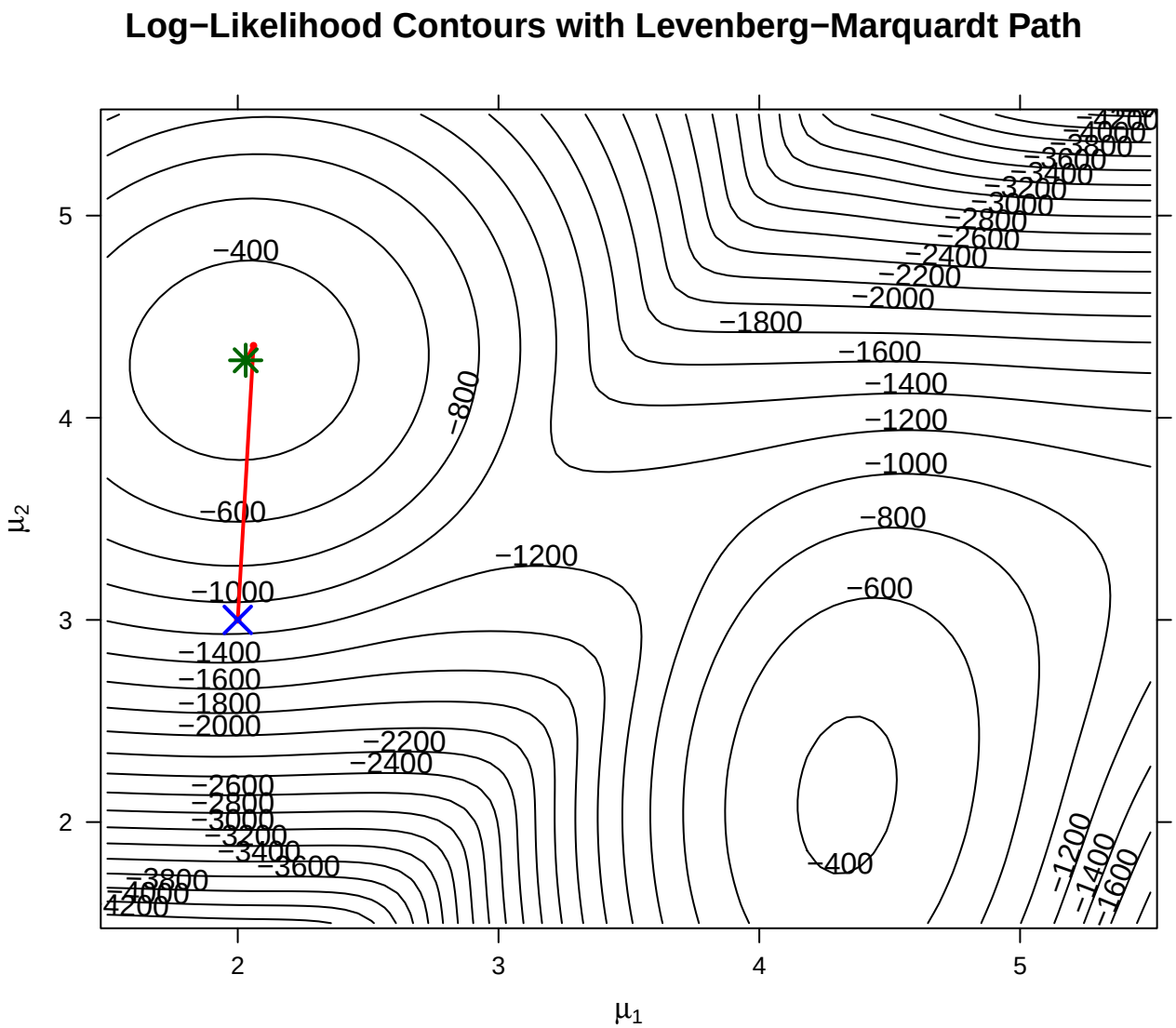
```
## Log-likelihood at MLE: -277.2088
```

```r
cat(sprintf("Iterations to convergence: %d\n", result$iter))
```

```
## Iterations to convergence: 5
```

```r
# Evaluate log-likelihood on a grid
mu1_seq   <- seq(1.5, 5.5, length.out = 80)
mu2_seq   <- seq(1.5, 5.5, length.out = 80)
grid_data <- expand.grid(mu1 = mu1_seq, mu2 = mu2_seq)
grid_data$ll <- mapply(function(m1, m2) log_lik(c(m1, m2)),
                       grid_data$mu1, grid_data$mu2)

lm_path <- result$history

cp <- contourplot(
  ll ~ mu1 * mu2, data = grid_data,
  cuts  = 20,
  xlab  = expression(mu[1]),
  ylab  = expression(mu[2]),
  main  = "Log-Likelihood Contours with Levenberg-Marquardt Path",
  panel = function(x, y, z, subscripts, ...) {
    panel.contourplot(x, y, z, subscripts, ...)
```

```
    panel.lines(lm_path[, 1], lm_path[, 2], col = "red",       lwd = 2)
    panel.points(lm_path[, 1], lm_path[, 2], pch = 16, col = "red", cex = 0.5)
    panel.points(lm_path[1, 1], lm_path[1, 2],
                 pch = 4,  col = "blue",       cex = 1.8, lwd = 2)
    panel.points(lm_path[nrow(lm_path), 1], lm_path[nrow(lm_path), 2],
                 pch = 8,  col = "darkgreen",  cex = 1.5, lwd = 2)
  }
)
print(cp)
```



The blue cross marks the starting point $(2, 3)$ and the green star marks the converged MLE. The algorithm follows the steepest-ascent direction initially (large λ) before transitioning to near-Newton steps as it approaches the optimum.

2 Problem 2: Linear Plateau Model

2.1 Part (a): Simple Linear Regression

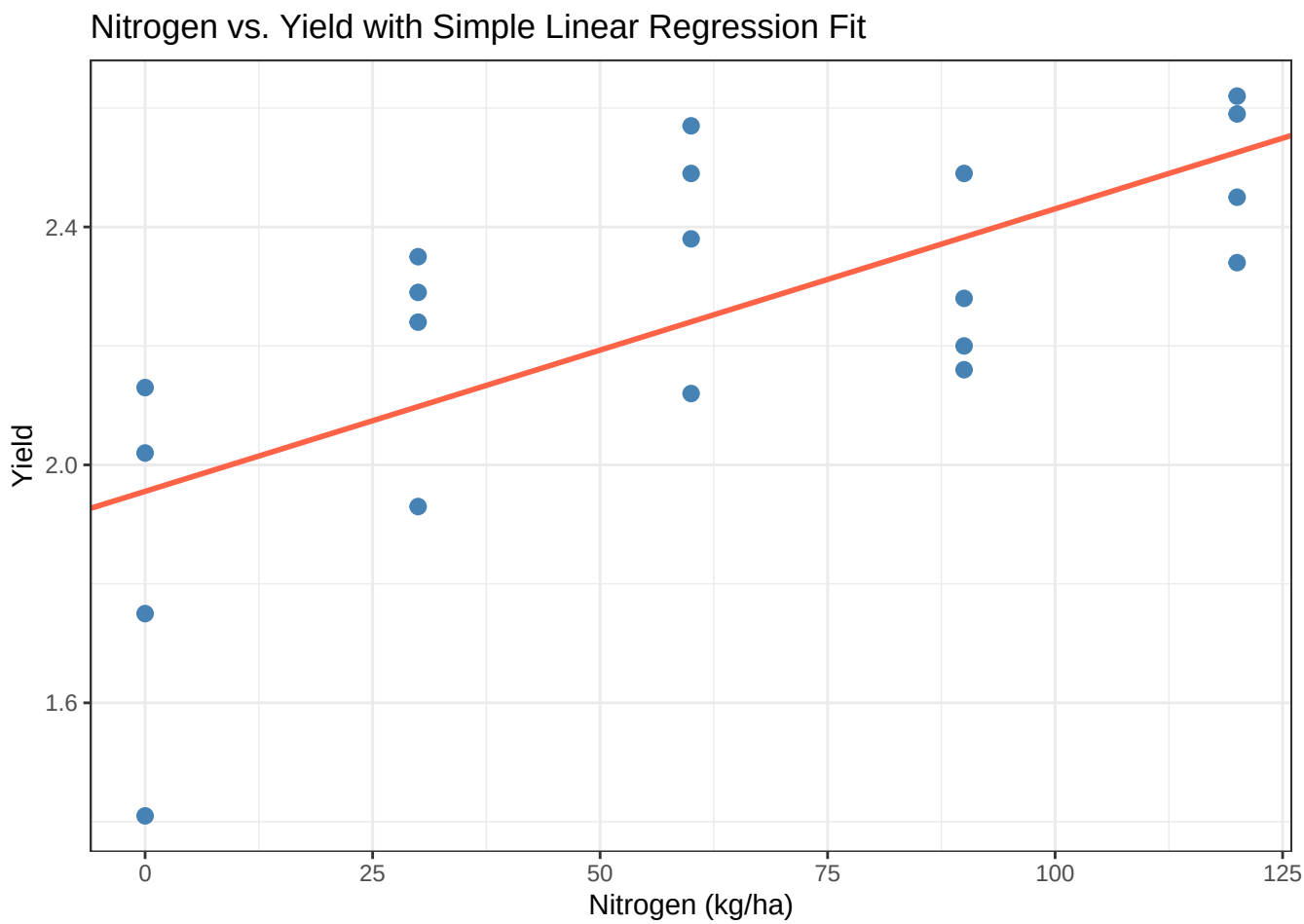
```r
nitrogen <- c(rep(0, 4), rep(30, 4), rep(60, 4), rep(90, 4), rep(120, 4))
yield    <- c(1.41, 1.75, 2.02, 2.13,
              1.93, 2.24, 2.29, 2.35,
              2.12, 2.38, 2.49, 2.57,
              2.16, 2.20, 2.28, 2.49,
              2.34, 2.45, 2.59, 2.62)
df_crop  <- data.frame(nitrogen = nitrogen, yield = yield)

lm_fit <- lm(yield ~ nitrogen, data = df_crop)
cat("Simple linear regression coefficients:\n")
```

```
## Simple linear regression coefficients:
```

```r
print(round(coef(lm_fit), 6))
```

```
## (Intercept)    nitrogen
##     1.95550     0.00475
```

```r
ggplot(df_crop, aes(x = nitrogen, y = yield)) +
  geom_point(size = 2.5, color = "steelblue") +
  geom_abline(intercept = coef(lm_fit)[1], slope = coef(lm_fit)[2],
              color = "tomato", linewidth = 1) +
  labs(title  = "Nitrogen vs. Yield with Simple Linear Regression Fit",
       x = "Nitrogen (kg/ha)", y = "Yield") +
  theme_bw()
```



2.2 Part (b): Gradient of the Model Function

The model function is $f(x, \theta) = \beta_0 + \beta_1 \min(x, N_{\max})$ with parameter vector $\theta = (\beta_0, \beta_1, N_{\max})^\top$. The partial derivatives are:

$$\frac{\partial f}{\partial \beta_0} = 1, \qquad \frac{\partial f}{\partial \beta_1} = \min(x, N_{\max}),$$

$$\frac{\partial f}{\partial N_{\max}} = \begin{cases} 0 & \text{if } x < N_{\max}, \\ \beta_1 & \text{if } x > N_{\max}. \end{cases}$$

When $x < N_{\max}$, the plateau has not been reached and $f = \beta_0 + \beta_1 x$ is independent of $N_{\max}$. When $x > N_{\max}$, the function evaluates at the plateau $f = \beta_0 + \beta_1 N_{\max}$, so $\partial f / \partial N_{\max} = \beta_1$. Combining, the gradient vector is

$$\nabla_\theta f(x, \theta) = \big(1,\ \min(x, N_{\max}),\ \beta_1 \cdot \mathbf{1}_{x > N_{\max}}\big)^\top.$$

2.3 Part (c): Gauss-Newton Fit via LPmodel

At each iteration the Gauss-Newton step minimizes a local linear approximation to the residuals. With residual vector $\mathbf{r}(\theta) = \mathbf{y} - \mathbf{f}(\theta)$ and Jacobian $J_{ij} = \partial f(x_i, \theta) / \partial \theta_j$, the update is

$$\Delta\theta = (J^\top J)^{-1} J^\top \mathbf{r}.$$

```r
LPmodel <- function(Y, X, tol = 1e-5, maxiter = 100) {
  # Starting values computed inside the function
  lm_start <- lm(Y ~ X)
  b0   <- as.numeric(coef(lm_start)[1])
  b1   <- as.numeric(coef(lm_start)[2])
  Nmax <- max(X) - 5

  del  <- Inf
  iter <- 0

  repeat {
    iter  <- iter + 1
    f_old <- b0 + b1 * pmin(X, Nmax)
    r     <- Y - f_old
    SS_old <- sum(r^2)

    # Jacobian (n x 3)
    J <- cbind(
      rep(1, length(X)),          # d f / d beta0
      pmin(X, Nmax),              # d f / d beta1
      b1 * as.numeric(X > Nmax) # d f / d Nmax
    )

    delta <- solve(t(J) %*% J, t(J) %*% r)
    b0   <- b0   + delta[1]
    b1   <- b1   + delta[2]
    Nmax <- Nmax + delta[3]

    f_new  <- b0 + b1 * pmin(X, Nmax)
    SS_new <- sum((Y - f_new)^2)
    del    <- abs(SS_new - SS_old)

    if (del < tol || iter >= maxiter) break
  }

  return(list(b0 = b0, b1 = b1, Nmax = Nmax, iter = iter, del = del))
}

lp_fit <- LPmodel(yield, nitrogen)

cat(sprintf("beta0 = %.5f\n", lp_fit$b0))
```

```
## beta0 = 1.94275
```

```r
cat(sprintf("beta1 = %.5f\n", lp_fit$b1))
```

```
## beta1 = 0.00517
```

```r
cat(sprintf("Nmax  = %.5f\n", lp_fit$Nmax))
```

```
## Nmax  = 107.68116
```

```r
cat(sprintf("Iterations: %d\n",     lp_fit$iter))
```

```
## Iterations: 3
```

```r
cat(sprintf("Final SS change: %.2e\n", lp_fit$del))
```

```
## Final SS change: 4.44e-16
```

```r
# Verify with nls
nls_fit <- nls(
  yield ~ b0 + b1 * pmin(nitrogen, Nmax),
  data  = df_crop,
  start = list(b0 = coef(lm_fit)[1], b1 = coef(lm_fit)[2], Nmax = max(nitrogen) - 5)
)
cat("\nVerification via nls():\n")
```

```
##
## Verification via nls():
```

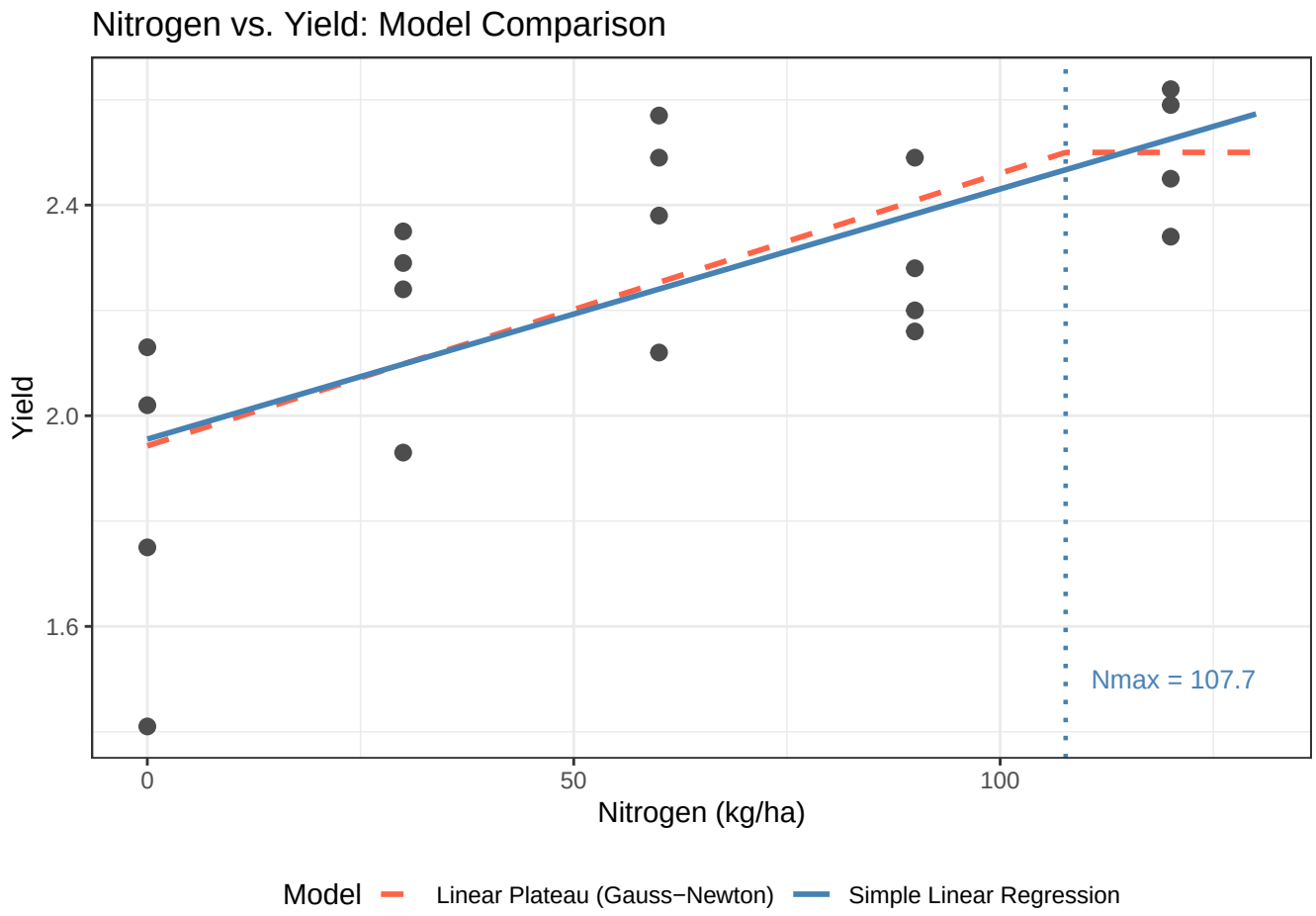
```r
print(round(coef(nls_fit), 5))
```

```
##        b0        b1      Nmax
##   1.94275   0.00518 107.68116
```

The Gauss-Newton estimates match the `nls` output, confirming correct implementation.

2.4 Part (d): Comparison Plot

```r
x_pred <- seq(0, 130, length.out = 300)
lm_pred <- coef(lm_fit)[1] + coef(lm_fit)[2] * x_pred
lp_pred <- lp_fit$b0 + lp_fit$b1 * pmin(x_pred, lp_fit$Nmax)

df_lines <- rbind(
  data.frame(nitrogen = x_pred, yield = lm_pred, Model = "Simple Linear Regression"),
  data.frame(nitrogen = x_pred, yield = lp_pred, Model = "Linear Plateau (Gauss-Newton)")
)

ggplot() +
  geom_point(data = df_crop, aes(x = nitrogen, y = yield),
             size = 2.5, color = "gray30") +
  geom_line(data = df_lines, aes(x = nitrogen, y = yield,
                                 color = Model, linetype = Model),
            linewidth = 1) +
  scale_color_manual(values = c("tomato", "steelblue")) +
  scale_linetype_manual(values = c("dashed", "solid")) +
  geom_vline(xintercept = lp_fit$Nmax, linetype = "dotted",
             color = "steelblue", linewidth = 0.8) +
  annotate("text", x = lp_fit$Nmax + 3, y = 1.5,
           label = paste0("Nmax = ", round(lp_fit$Nmax, 1)),
           hjust = 0, color = "steelblue", size = 3.5) +
  labs(title   = "Nitrogen vs. Yield: Model Comparison",
       x = "Nitrogen (kg/ha)", y = "Yield",
       color = "Model", linetype = "Model") +
  theme_bw() +
  theme(legend.position = "bottom")
```



The linear plateau model captures the diminishing returns at high nitrogen levels, while simple linear regression continues to predict increasing yields beyond the plateau. The estimated plateau is reached at approximately $N_{\max} \approx 107.7$ kg/ha.

3 Problem 3: Challenger O-Ring Logistic Regression

3.1 Part (a): Gradient and Hessian of the Log-Likelihood

Let $y_i = \mathbf{1}(\text{ndo}_i > 0)$ be the binary damage indicator and $\pi_i = P(y_i = 1) = \sigma(\beta_0 + \beta_1 x_i)$, where $\sigma(\eta) = (1 + e^{-\eta})^{-1}$. The log-likelihood is

$$\ell(\beta_0, \beta_1) = \sum_{i=1}^{n} \big[y_i \log \pi_i + (1 - y_i) \log(1 - \pi_i) \big].$$

Using $\partial \pi_i / \partial \eta_i = \pi_i(1 - \pi_i)$ with $\eta_i = \beta_0 + \beta_1 x_i$, the **gradient** is

$$\frac{\partial \ell}{\partial \beta_0} = \sum_{i=1}^{n} (y_i - \pi_i), \qquad \frac{\partial \ell}{\partial \beta_1} = \sum_{i=1}^{n} x_i (y_i - \pi_i).$$

In matrix form, with design matrix $X = [\mathbf{1}, \mathbf{x}]$ and weight vector $w_i = \pi_i(1 - \pi_i)$:

$$\nabla \ell = X^\top (\mathbf{y} - \boldsymbol{\pi}).$$

The **Hessian** is

$$H = \frac{\partial^2 \ell}{\partial \beta \, \partial \beta^\top} = -X^\top W X,$$

where $W = \text{diag}(\pi_i(1 - \pi_i))$. Explicitly,

$$\frac{\partial^2 \ell}{\partial \beta_0^2} = -\sum_i \pi_i(1 - \pi_i), \quad \frac{\partial^2 \ell}{\partial \beta_1^2} = -\sum_i x_i^2 \pi_i(1 - \pi_i), \quad \frac{\partial^2 \ell}{\partial \beta_0 \partial \beta_1} = -\sum_i x_i \pi_i(1 - \pi_i).$$

H is negative semi-definite, guaranteeing concavity of ℓ.

3.2 Part (b): Newton-Raphson via oringNR

The Newton-Raphson update $\beta^{(t+1)} = \beta^{(t)} - H^{-1} \nabla \ell$ is iterated from starting values $\hat{\beta}_0^{(0)} = \log(\bar{y}/(1 - \bar{y}))$ and $\hat{\beta}_1^{(0)} = 0$.

```r
oringNR <- function(filename) {
  data <- read.csv(filename)

  # Binary damage indicator
  y <- as.numeric(data$ndo > 0)
  x <- data$temp
  n <- length(y)

  # Starting values
  ybar <- mean(y)
  b0   <- log(ybar / (1 - ybar))
  b1   <- 0

  # History matrix: columns = (iteration, beta0, beta1)
  histOR <- matrix(c(0, b0, b1), nrow = 1)
```

```r
  tol     <- 1e-8
  maxiter <- 100

  for (iter in seq_len(maxiter)) {
    eta  <- b0 + b1 * x
    pi_i <- 1 / (1 + exp(-eta))
    w    <- pi_i * (1 - pi_i)

    # Gradient
    g <- c(sum(y - pi_i), sum(x * (y - pi_i)))

    # Hessian
    H <- matrix(c(-sum(w),       -sum(x * w),
                  -sum(x * w),   -sum(x^2 * w)),
                nrow = 2, byrow = TRUE)

    # Newton-Raphson step: beta_new = beta - H^{-1} g
    delta <- solve(H, g)
    b0    <- b0 - delta[1]
    b1    <- b1 - delta[2]

    histOR <- rbind(histOR, c(iter, b0, b1))

    if (max(abs(delta)) < tol) break
  }

  colnames(histOR) <- c("iteration", "beta0", "beta1")
  return(histOR)
}
```

```r
histOR <- oringNR("shuttle.csv")

cat("Newton-Raphson convergence history:\n")
```

```
## Newton-Raphson convergence history:
```

```r
print(round(histOR, 6))
```

```
##      iteration     beta0     beta1
## [1,]         0 -0.826679  0.000000
## [2,]         1 11.455634 -0.176558
## [3,]         2 14.322669 -0.221199
## [4,]         3 15.012823 -0.231711
## [5,]         4 15.042850 -0.232162
## [6,]         5 15.042902 -0.232163
## [7,]         6 15.042902 -0.232163
```

```r
mle_b0 <- tail(histOR[, "beta0"], 1)
mle_b1 <- tail(histOR[, "beta1"], 1)
n_iter <- nrow(histOR) - 1

cat(sprintf("\nMLE: beta0 = %.4f, beta1 = %.4f\n", mle_b0, mle_b1))
```

```
##
## MLE: beta0 = 15.0429, beta1 = -0.2322
```

```r
cat(sprintf("Iterations to convergence: %d\n", n_iter))
```

```
## Iterations to convergence: 6
```

3.3 Part (c): Bootstrap Standard Errors

```r
shuttle  <- read.csv("shuttle.csv")
y_full   <- as.numeric(shuttle$ndo > 0)
x_full   <- shuttle$temp
n_full   <- length(y_full)

B           <- 2000
boot_coefs  <- matrix(NA, B, 2)

for (b in seq_len(B)) {
  idx <- sample(n_full, replace = TRUE)
  y_b <- y_full[idx]
  x_b <- x_full[idx]

  ybar_b <- mean(y_b)
  if (ybar_b <= 0 || ybar_b >= 1) next  # skip degenerate samples

  b0_b <- log(ybar_b / (1 - ybar_b))
  b1_b <- 0

  for (iter in seq_len(100)) {
    eta_b <- b0_b + b1_b * x_b
    pi_b  <- 1 / (1 + exp(-eta_b))
    w_b   <- pi_b * (1 - pi_b)
    g_b   <- c(sum(y_b - pi_b), sum(x_b * (y_b - pi_b)))
    H_b   <- matrix(c(-sum(w_b),      -sum(x_b * w_b),
                      -sum(x_b * w_b), -sum(x_b^2 * w_b)),
                    nrow = 2, byrow = TRUE)
    delta_b <- tryCatch(solve(H_b, g_b), error = function(e) c(NA, NA))
    if (any(is.na(delta_b))) break
    b0_b <- b0_b - delta_b[1]
    b1_b <- b1_b - delta_b[2]
    if (max(abs(delta_b)) < 1e-8) break
  }
  boot_coefs[b, ] <- c(b0_b, b1_b)
}

se_b0 <- sd(boot_coefs[, 1], na.rm = TRUE)
se_b1 <- sd(boot_coefs[, 2], na.rm = TRUE)

cat(sprintf("Bootstrap SE for beta0: %.4f\n", se_b0))
```

```
## Bootstrap SE for beta0: 234.8188
```

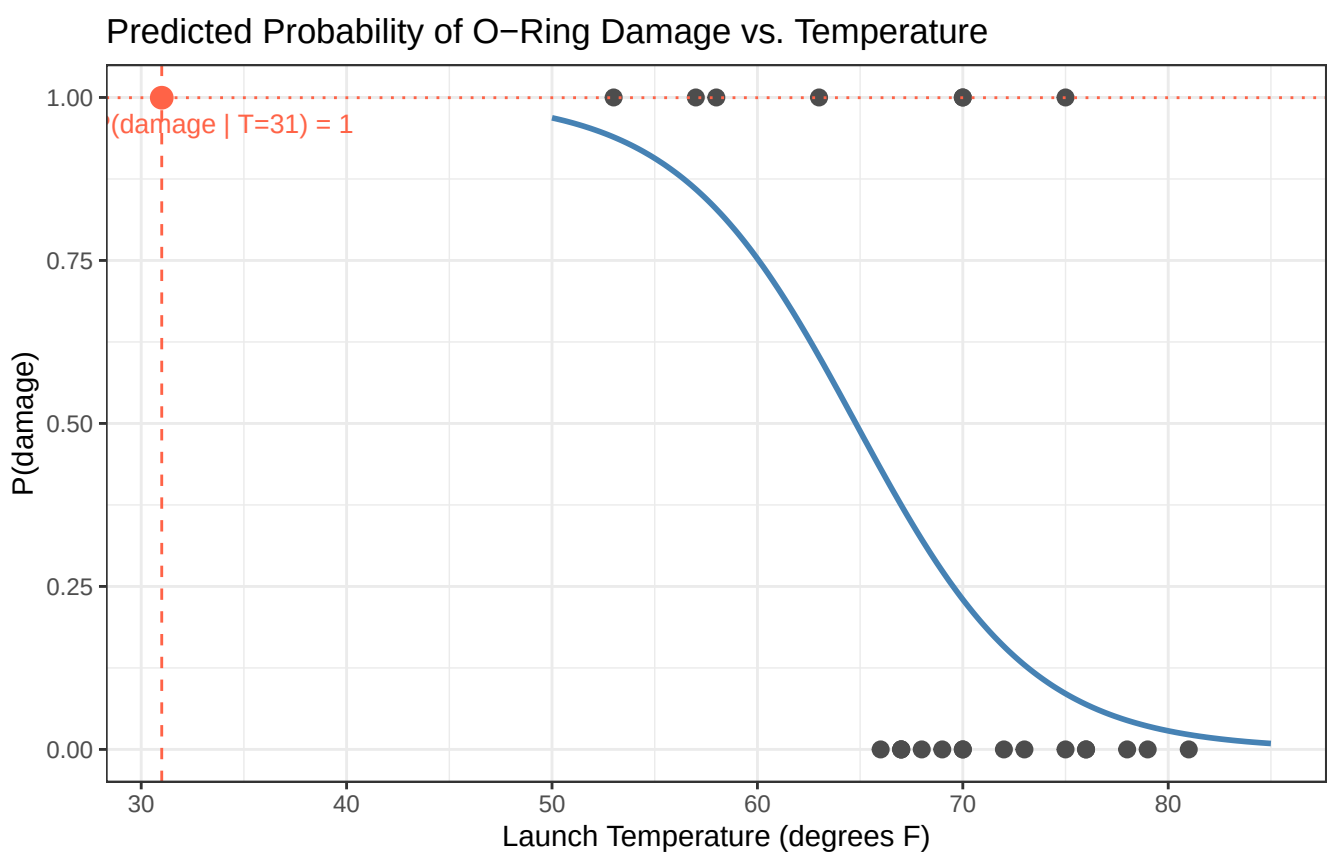
```r
cat(sprintf("Bootstrap SE for beta1: %.4f\n", se_b1))
```

```
## Bootstrap SE for beta1: 3.6530
```

The bootstrap standard errors characterize the sampling variability of the MLEs without requiring analytical variance formulas.

3.4 Part (d): Probability of O-Ring Damage and Prediction at 31 degrees F

```
temp_seq   <- seq(50, 85, length.out = 300)
pi_seq     <- 1 / (1 + exp(-(mle_b0 + mle_b1 * temp_seq)))
pi_31      <- 1 / (1 + exp(-(mle_b0 + mle_b1 * 31)))

df_curve   <- data.frame(temp = temp_seq, pi = pi_seq)
df_obs     <- data.frame(temp = x_full,   pi = y_full)

ggplot() +
  geom_line(data = df_curve, aes(x = temp, y = pi),
            color = "steelblue", linewidth = 1) +
  geom_point(data = df_obs, aes(x = temp, y = pi),
             color = "gray30", size = 2.5) +
  geom_vline(xintercept = 31, linetype = "dashed", color = "tomato") +
  geom_hline(yintercept = pi_31, linetype = "dotted", color = "tomato") +
  annotate("point", x = 31, y = pi_31, color = "tomato", size = 3.5) +
  annotate("text",
           x = 34, y = pi_31 - 0.04,
           label = paste0("P(damage | T=31) = ", round(pi_31, 3)),
           color = "tomato", size = 3.5) +
  scale_y_continuous(limits = c(0, 1)) +
  labs(title = "Predicted Probability of O-Ring Damage vs. Temperature",
       x = "Launch Temperature (degrees F)", y = "P(damage)") +
  theme_bw()
```



```r
cat(sprintf("Predicted probability of damage at 31 degrees F: %.4f\n", pi_31))
```

```
## Predicted probability of damage at 31 degrees F: 0.9996
```

The model predicts an extremely high probability of o-ring damage at 31°F, the temperature at the time of the Challenger launch. The fitted curve shows a sharp increase in failure probability below approximately 65°F, consistent with the physical mechanism of o-ring stiffening at low temperatures.

4 Problem 4

Claim. The conditional distribution of SY given $Z < \phi(Y)$ is $N(0,1)$, where $S \sim U\{-1,1\}$, $Y \sim \mathrm{Exp}(1)$, $Z \mid Y \sim U\Big(0,\, e^{-Y}\sqrt{2e/\pi}\Big)$, and ϕ is the standard normal density.

Proof. Let $W = SY$. Since $S \in \{-1,1\}$ and $Y > 0$, the random variable W takes values in $\mathbb{R} \setminus \{0\}$ with $|W| = Y$.

For any $w \neq 0$ consider the joint probability element $P(W \in dw,\, Z < \phi(Y))$. By the law of total probability, conditioning on $Y = |w|$:

$$P(W \in dw,\, Z < \phi(Y)) = \frac{1}{2} \cdot f_Y(|w|) \cdot P(Z < \phi(|w|) \mid Y = |w|)\, d|w|.$$

Substituting $f_Y(|w|) = e^{-|w|}$ and the conditional uniform probability

$$P(Z < \phi(|w|) \mid Y = |w|) = \frac{\phi(|w|)}{e^{-|w|}\sqrt{2e/\pi}},$$

the $e^{-|w|}$ factors cancel to give

$$P(W \in dw,\, Z < \phi(Y)) = \frac{\phi(|w|)}{2\sqrt{2e/\pi}}\, d|w| = \frac{\phi(w)}{2\sqrt{2e/\pi}}\, dw,$$

using the symmetry $\phi(|w|) = \phi(w)$ in the last step.

Therefore the conditional density of $W = SY$ given $Z < \phi(Y)$ is

$$f_{W|Z<\phi(Y)}(w) = \frac{\phi(w)/(2\sqrt{2e/\pi})}{\displaystyle\int_{-\infty}^{\infty} \phi(w)/(2\sqrt{2e/\pi})\, dw} = \frac{\phi(w)}{1} = \phi(w),$$

since $\int_{-\infty}^{\infty} \phi(w)\, dw = 1$. Hence $SY \mid Z < \phi(Y) \sim N(0,1)$. ■

Remark. The key step is that the exponential density $f_Y(y) = e^{-y}$ and the upper bound of $Z \mid Y$, namely $e^{-y}\sqrt{2e/\pi}$, share the same e^{-y} factor, which cancels in the ratio. This is precisely the structure that makes the scheme a valid accept-reject sampler for $N(0,1)$.

5 Problem 5

Claim. Let (A, B) be uniformly distributed on the unit disk $\{(a, b) : a^2 + b^2 \leq 1\}$, and let (S, Ψ) be its polar coordinates with $S = A^2 + B^2$ and $\Psi = \arctan(B/A) \in [0, 2\pi)$. Then $S \sim U(0, 1)$, $\Psi \sim U(0, 2\pi)$, and S and Ψ are independent.

Proof. The joint density of (A, B) is $f_{A,B}(a, b) = 1/\pi$ on the unit disk.

Step 1: Change to polar coordinates (R, Ψ). Writing $A = R\cos\Psi$ and $B = R\sin\Psi$ with $R \in [0, 1]$ and $\Psi \in [0, 2\pi)$, the Jacobian is $|\partial(A, B)/\partial(R, \Psi)| = R$, so

$$f_{R,\Psi}(r, \psi) = \frac{1}{\pi} \cdot r, \qquad r \in [0, 1],\ \psi \in [0, 2\pi).$$

Step 2: Change to (S, Ψ) with $S = R^2$. Since $R = S^{1/2}$, the Jacobian of this one-dimensional transformation is $|dR/dS| = \frac{1}{2}S^{-1/2}$, giving

$$f_{S,\Psi}(s, \psi) = f_{R,\Psi}(s^{1/2}, \psi) \cdot \tfrac{1}{2}s^{-1/2} = \frac{1}{\pi} \cdot s^{1/2} \cdot \tfrac{1}{2}s^{-1/2} = \frac{1}{2\pi},$$

for $s \in [0, 1]$ and $\psi \in [0, 2\pi)$.

Step 3: Conclude. Because $f_{S,\Psi}(s, \psi) = 1/(2\pi)$ is constant on $[0, 1] \times [0, 2\pi)$, it factors as

$$f_{S,\Psi}(s, \psi) = \underbrace{1}_{f_S(s)} \cdot \underbrace{\frac{1}{2\pi}}_{f_\Psi(\psi)}.$$

Thus the marginal densities are $f_S(s) = 1$ on $[0, 1]$ and $f_\Psi(\psi) = 1/(2\pi)$ on $[0, 2\pi)$, so $S \sim U(0, 1)$ and $\Psi \sim U(0, 2\pi)$. Since the joint density factors into the product of the marginals, S and Ψ are independent. ■

6 Appendix: Session Information

```r
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=English_United States.utf8
## [2] LC_CTYPE=English_United States.utf8
## [3] LC_MONETARY=English_United States.utf8
## [4] LC_NUMERIC=C
## [5] LC_TIME=English_United States.utf8
##
## time zone: America/New_York
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
## [1] lattice_0.22-7 ggplot2_4.0.1
##
## loaded via a namespace (and not attached):
##  [1] vctrs_0.6.5        cli_3.6.5          knitr_1.51         rlang_1.1.7
##  [5] xfun_0.55          generics_0.1.4     S7_0.2.1           labeling_0.4.3
##  [9] glue_1.8.0         htmltools_0.5.9    scales_1.4.0       rmarkdown_2.30
## [13] grid_4.5.2         evaluate_1.0.5     tibble_3.3.1       fastmap_1.2.0
## [17] yaml_2.3.12        lifecycle_1.0.5    compiler_4.5.2     dplyr_1.1.4
## [21] RColorBrewer_1.1-3 pkgconfig_2.0.3    farver_2.1.2       digest_0.6.39
## [25] R6_2.6.1           tidyselect_1.2.1   pillar_1.11.1      magrittr_2.0.4
## [29] withr_3.0.2        tools_4.5.2        gtable_0.3.6
```